Model.

```
package models

import (
"time"

"github.com/google/uuid"
"gorm.io/gorm"
)

// Enum types for better type safety
type PartnerType string
type AgentType string
type StatusType string

const (
// Partner types
PartnerTypeInsurance PartnerType = "insurance"
PartnerTypeBank PartnerType = "bank"
PartnerTypeDealer PartnerType = "dealer"
PartnerTypeOther PartnerType = "other"

// Agent types
AgentTypeIndividual AgentType = "individual"
AgentTypeCorporate AgentType = "corporate"

// Status types
StatusActive StatusType = "active"
StatusInactive StatusType = "inactive"
StatusPending StatusType = "pending"
StatusApproved StatusType = "approved"
StatusRejected StatusType = "rejected"
StatusPaid StatusType = "paid"
StatusUnpaid StatusType = "unpaid"
)

// Customer represents customer data for valuation service
type Customer struct {
ID string `json:"id" gorm:"type:uuid;primary_key"`
Email string `json:"email" gorm:"not null;unique"`
Phone string `json:"phone" gorm:"not null;unique"`
FirstName string `json:"first_name" gorm:"not null"`
LastName string `json:"last_name" gorm:"not null"`
Status string `json:"status" gorm:"not null"`
```

```
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt *gorm.DeletedAt `json:"deleted_at,omitempty" gorm:"index"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:CustomerID"`
VehicleInsurances []VehicleInsurances `json:"vehicle_insurances"
gorm:"foreignKey:CustomerID"`
}

// Partners represents partner organizations
type Partners struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
PartnerType PartnerType `json:"partner_type" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Agents []Agents `json:"agents" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:PartnerID"`
}

// Agents represents partner agents
type Agents struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
AgentType AgentType `json:"agent_type" gorm:"not null"`
PartnerID string `json:"partner_id" gorm:"not null"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
CompanyPhone string `json:"company_phone" gorm:"not null"`
CompanyEmail string `json:"company_email" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
```

```
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Partner Partners `json:"partner" gorm:"foreignKey:PartnerID"`
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:AgentID"`
}

// ValuationTypes represents different types of valuations
type ValuationTypes struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null;unique"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
DeletedAt time.Time `json:"deleted_at"`
}

// Branches represents company branches
type Branches struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
Location string `json:"location" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`
}

// VehicleInsurances represents vehicle insurance information
type VehicleInsurances struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InsuranceCompany string `json:"insurance_company" gorm:"not null"`
PolicyNumber string `json:"policy_number" gorm:"unique;not null"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
CustomerID string `json:"customer_id" gorm:"not null"`
ExpiryDate time.Time `json:"expiry_date" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
}
```

```
// Valuer represents professional valuers
type Valuer struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
Name string `json:"name" gorm:"not null"`
Phone string `json:"phone" gorm:"not null"`
Location string `json:"location" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:active"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
ValuationRequests []ValuationRequests `json:"valuation_requests"
gorm:"foreignKey:ValuerID"`
}

// ValuationRequests represents valuation requests
type ValuationRequests struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
VehicleID string `json:"vehicle_id" gorm:"not null"`
PartnerID *string `json:"partner_id,omitempty"`
AgentID *string `json:"agent_id,omitempty"`
CustomerID string `json:"customer_id" gorm:"not null"`
LocationAddress string `json:"location_address" gorm:"not null"`
LocationCoordinates *string `json:"location_coordinates,omitempty"`
ContactPersonName string `json:"contact_person_name" gorm:"not null"`
ContactPersonPhone string `json:"contact_person_phone" gorm:"not null"`
ValuerID *string `json:"valuer_id,omitempty"`
ApproverID *string `json:"approver_id,omitempty"`
Status StatusType `json:"status" gorm:"default:pending"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
Partner *Partners `json:"partner,omitempty" gorm:"foreignKey:PartnerID"`
Agent *Agents `json:"agent,omitempty" gorm:"foreignKey:AgentID"`
Customer Customer `json:"customer" gorm:"foreignKey:CustomerID"`
Valuer *Valuer `json:"valuer,omitempty" gorm:"foreignKey:ValuerID"`
ValuationInvoices []ValuationInvoices `json:"valuation_invoices"
gorm:"foreignKey:ValuationRequestID"`
InspectionAnswers []InspectionAnswers `json:"inspection_answers"
gorm:"foreignKey:ValuationRequestID"`
}

// ValuationInvoices represents invoices for valuation services
```

```
type ValuationInvoices struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
ValuationRequestID string `json:"valuation_request_id" gorm:"not null"`
InvoiceNumber string `json:"invoice_number" gorm:"unique;not null"`
AmountBilled float64 `json:"amount_billed" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:unpaid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
ValuationRequest ValuationRequests `json:"valuation_request"
gorm:"foreignKey:ValuationRequestID"`
//ValuationPayments []ValuationPayments `json:"valuation_payments"
gorm:"foreignKey:InvoiceNumber;references:InvoiceNumber"`
}

// ValuationPayments represents payments for valuation invoices
type ValuationPayments struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
InvoiceNumber string `json:"invoice_number" gorm:"not null"`
ReceiptNumber string `json:"receipt_number" gorm:"unique;not null"`
AmountPaid float64 `json:"amount_paid" gorm:"not null"`
Status StatusType `json:"status" gorm:"default:paid"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships - Note: Reference by InvoiceNumber instead of ID
//ValuationInvoice ValuationInvoices `json:"valuation_invoice"
gorm:"foreignKey:InvoiceNumber;references:InvoiceNumber"`
}

// InspectionSections represents different sections of vehicle inspection
type InspectionSections struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
SectionName string `json:"section_name" gorm:"not null;unique"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
InspectionQuestions []InspectionQuestions `json:"inspection_questions"
gorm:"foreignKey:SectionID"`
}

// InspectionQuestions represents questions within each inspection section
```

```
type InspectionQuestions struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
SectionID string `json:"section_id" gorm:"not null"`
Question string `json:"question" gorm:"not null"`
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
InspectionSection InspectionSections `json:"inspection_section"
gorm:"foreignKey:SectionID"`
InspectionAnswers []InspectionAnswers `json:"inspection_answers"
gorm:"foreignKey:QuestionID"`
}

// InspectionAnswers represents answers to inspection questions for specific
valuation requests
type InspectionAnswers struct {
ID string `json:"id" gorm:"type:uuid;primary_key;default:gen_random_uuid()"`
QuestionID string `json:"question_id" gorm:"not null"`
ValuationRequestID string `json:"valuation_request_id" gorm:"not null"`
Answer bool `json:"answer" gorm:"not null"` // true or false
Description string `json:"description,omitempty"` // Optional additional description
CreatedAt time.Time `json:"created_at"`
UpdatedAt time.Time `json:"updated_at"`

// Relationships
InspectionQuestion InspectionQuestions `json:"inspection_question"
gorm:"foreignKey:QuestionID"`
ValuationRequest ValuationRequests `json:"valuation_request"
gorm:"foreignKey:ValuationRequestID"`
}

// BeforeCreate hooks for UUID generation
func (c *Customer) BeforeCreate(tx *gorm.DB) error {
if c.ID == "" {
c.ID = uuid.New().String()
}
return nil
}

func (p *Partners) BeforeCreate(tx *gorm.DB) error {
if p.ID == "" {
p.ID = uuid.New().String()
}
```

```
return nil
}

func (a *Agents) BeforeCreate(tx *gorm.DB) error {
if a.ID == "" {
a.ID = uuid.New().String()
}
return nil
}

func (vt *ValuationTypes) BeforeCreate(tx *gorm.DB) error {
if vt.ID == "" {
vt.ID = uuid.New().String()
}
return nil
}

func (b *Branches) BeforeCreate(tx *gorm.DB) error {
if b.ID == "" {
b.ID = uuid.New().String()
}
return nil
}

func (vi *VehicleInsurances) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
vi.ID = uuid.New().String()
}
return nil
}

func (v *Valuer) BeforeCreate(tx *gorm.DB) error {
if v.ID == "" {
v.ID = uuid.New().String()
}
return nil
}

func (vr *ValuationRequests) BeforeCreate(tx *gorm.DB) error {
if vr.ID == "" {
vr.ID = uuid.New().String()
}
return nil
}
```

```
func (vi *ValuationInvoices) BeforeCreate(tx *gorm.DB) error {
if vi.ID == "" {
vi.ID = uuid.New().String()
}
return nil
}

func (vp *ValuationPayments) BeforeCreate(tx *gorm.DB) error {
if vp.ID == "" {
vp.ID = uuid.New().String()
}
return nil
}

// BeforeCreate hooks for new inspection models
func (is *InspectionSections) BeforeCreate(tx *gorm.DB) error {
if is.ID == "" {
is.ID = uuid.New().String()
}
return nil
}

func (iq *InspectionQuestions) BeforeCreate(tx *gorm.DB) error {
if iq.ID == "" {
iq.ID = uuid.New().String()
}
return nil
}

func (ia *InspectionAnswers) BeforeCreate(tx *gorm.DB) error {
if ia.ID == "" {
ia.ID = uuid.New().String()
}
return nil
}
```

Main.go

```
package main

import (
"context"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/
rabbitmq"
"log"
"net/http"
"os"
"os/signal"
"syscall"
"time"

_ "github.com/Automobile-Association-of-Kenya/aa-core-system/docs/vehicle-service"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/
database"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/shared/redis"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-
management/config"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/handlers"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/models"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/repositories"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/routes"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/vehicle-
service/services"
authclient "github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/auth-
client"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/middleware"
"github.com/gin-gonic/gin"
"github.com/joho/godotenv"
"gorm.io/gorm"
)

// @title AA Core System - Vehicle Service API
// @version 1.0
// @description Vehicle management service for AA Core System
// @contact.name API Support
// @contact.email support@aa.co.ke
```

```
// @host localhost:8082
// @BasePath /api/v1
// @securityDefinitions.apikey BearerAuth
// @in header
// @name Authorization
func main() {
// Load environment variables
if err := godotenv.Load(); err != nil {
log.Println("No .env file found, using system environment variables")
}

// Load configuration
cfg, err := config.LoadConfig("./configs")
if err != nil {
log.Fatalf("Failed to load config: %v", err)
}

// Set Gin mode
gin.SetMode(cfg.Server.Mode)

// Initialize database (only vehicle database needed)
dbConfig := database.Config{
Host: cfg.Database.Host,
Port: cfg.Database.Port,
User: cfg.Database.User,
Password: cfg.Database.Password,
DBName: cfg.Database.VehicleDatabaseName, // Separate database
SSLMode: cfg.Database.SSLMode,
MaxOpenConns: cfg.Database.MaxOpenConns,
MaxIdleConns: cfg.Database.MaxIdleConns,
ConnMaxLifetime: cfg.Database.ConnMaxLifetime,
}

db, err := database.NewPostgresConnection(dbConfig)
if err != nil {
log.Fatalf("Failed to connect to database: %v", err)
}

// Run migrations
if err := runMigrations(db); err != nil {
log.Fatalf("Failed to run migrations: %v", err)
}

// Initialize Redis for caching
```

```
redisConfig := redis.Config{
Host: cfg.Redis.Host,
Port: cfg.Redis.Port,
Password: cfg.Redis.Password,
DB: 2, // Different Redis DB for vehicle service
PoolSize: cfg.Redis.PoolSize,
}

redisClient, err := redis.NewRedisClient(redisConfig)
if err != nil {
log.Fatalf("Failed to connect to Redis: %v", err)
}

// Initialize RabbitMQ (replacing Kafka)
rabbitmqURL := os.Getenv("RABBITMQ_URL")
if rabbitmqURL == "" {
rabbitmqURL = cfg.RabbitMQ.URL
}

rabbitmqConfig := rabbitmq.Config{
URL: rabbitmqURL,
ExchangeName: cfg.RabbitMQ.ExchangeName,
Durable: cfg.RabbitMQ.Durable,
AutoDelete: cfg.RabbitMQ.AutoDelete,
PrefetchCount: cfg.RabbitMQ.PrefetchCount,
ReconnectDelay: cfg.RabbitMQ.ReconnectDelay,
MaxReconnectTime: cfg.RabbitMQ.MaxReconnectTime,
}

rabbitmqProducer, err := rabbitmq.NewProducer(rabbitmqConfig)
if err != nil {
log.Fatalf("Failed to connect to RabbitMQ: %v", err)
}
defer rabbitmqProducer.Close()

// Initialize Auth Service Client
authServiceURL := os.Getenv("AUTH_SERVICE_URL")
if authServiceURL == "" {
authServiceURL = cfg.Server.AuthUrl // Your auth service port
}

log.Printf("🔐 Configuring auth client with URL: %s", authServiceURL)

authClientConfig := authclient.Config{
```

```
AuthServiceURL: authServiceURL,
ServiceID: "vehicle-service",
ServiceVersion: "1.0",
SecretKey: cfg.JWT.Secret, // Same secret as auth service
Timeout: 5 * time.Second,
RedisClient: redisClient,
FallbackMode: true, // Enable fallback for resilience
}

authClient := authclient.NewAuthClient(authClientConfig)

// Test auth service connection
log.Printf("🔍 Testing auth service connection...")
if authClient.IsHealthy() {
log.Printf("✅ Auth service is healthy")
} else {
log.Printf("❌ Auth service is not responding")
}

// Initialize auth middleware
authMiddleware := middleware.NewAuthServiceMiddleware(authClient, true)

// Initialize repositories (only vehicle repo needed)
vehicleRepo := repositories.NewVehicleRepository(db)

// Initialize services
vehicleService := services.NewVehicleService(vehicleRepo, rabbitmqProducer)

// Initialize handlers
vehicleHandler := handlers.NewVehicleHandler(vehicleService)

// Initialize Gin router
router := gin.New()
router.Use(gin.Recovery())

// Setup routes with auth service middleware
routes.SetupRoutes(router, vehicleHandler, authMiddleware)

// Start Kafka consumers in background
go startRabbitMQConsumers(rabbitmqConfig, "vehicle-service", vehicleService)

// Start auth service health monitoring
go monitorAuthService(authClient)
```

```
// Seed initial data
go func() {
time.Sleep(5 * time.Second) // Wait for service to be ready
if err := seedInitialData(vehicleService); err != nil {
log.Printf("Failed to seed initial data: %v", err)
}
}()

port := os.Getenv("PORT")
if port == "" {
port = "8082" // Your vehicle service port
}

// Create HTTP server
srv := &http.Server{
Addr: ":" + port,
Handler: router,
ReadTimeout: cfg.Server.ReadTimeout,
WriteTimeout: cfg.Server.WriteTimeout,
}

// Start server in a goroutine
go func() {
log.Printf("Starting vehicle service on port %s", port)
log.Printf("Using auth service at: %s", authServiceURL)
if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
log.Fatalf("Failed to start server: %v", err)
}
}()

// Wait for interrupt signal to gracefully shutdown the server
quit := make(chan os.Signal, 1)
signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
<-quit

log.Println("Shutting down vehicle service...")

ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer cancel()

if err := srv.Shutdown(ctx); err != nil {
log.Fatalf("Server forced to shutdown: %v", err)
}
```

```
log.Println("Vehicle service shutdown complete")
}

func runMigrations(db *gorm.DB) error {
log.Println("Running vehicle service migrations...")

err := db.AutoMigrate(
&models.Customer{},
&models.VehicleMake{},
&models.VehicleModel{},
&models.CustomerVehicle{},
&models.KraCSRP{}, // Added KRA CSRP model
)

if err != nil {
return err
}

log.Println("Vehicle service migrations completed successfully")
return nil
}

func startRabbitMQConsumers(rabbitmqConfig rabbitmq.Config, consumerGroup
string, vehicleService *services.VehicleService) {
consumer, err := rabbitmq.NewConsumer(rabbitmqConfig, consumerGroup)
if err != nil {
log.Fatalf("Failed to create RabbitMQ consumer: %v", err)
return
}

// Handle user events
go func() {
err := consumer.ConsumeEvents("user_events", consumerGroup, func(event
rabbitmq.Event) error {
switch event.Type {
case "user.registered":
return vehicleService.HandleCustomerCreated(event)
case "user.email_verified":
return vehicleService.HandleCustomerVerified(event)
case "user.phone_verified":
return vehicleService.HandlePhoneVerified(event)
default:
log.Printf("Payment service: Unhandled event type: %s", event.Type)
}
```

```
return nil
})
if err != nil {
log.Printf("Failed to consume user events in payment service: %v", err)
}
}()

// You can add more consumers for different event types here
// For example, to consume vehicle events or valuation events:
/*
go func() {
err := consumer.ConsumeEvents("vehicle_events", consumerGroup, func(event
rabbitmq.Event) error {
switch event.Type {
case "vehicle.registered":
// Handle vehicle registration for insurance payments
return paymentService.HandleVehicleRegistered(event)
default:
log.Printf("Payment service: Unhandled vehicle event type: %s", event.Type)
}
return nil
})
if err != nil {
log.Printf("Failed to consume vehicle events in payment service: %v", err)
}
}()
*/

log.Println("Payment service RabbitMQ consumers started successfully")
}

func monitorAuthService(authClient *authclient.AuthClient) {
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()

for range ticker.C {
if !authClient.IsHealthy() {
log.Printf("Warning: Auth service is not responding")
}
}
}

func seedInitialData(vehicleService *services.VehicleService) error {
log.Println("Seeding initial vehicle data...")
```

```
// Popular vehicle makes
makes := []services.CreateMakeRequest{
{Name: "Toyota", Country: "Japanese"},
{Name: "Honda", Country: "Japanese"},
{Name: "Nissan", Country: "Japanese"},
{Name: "Mazda", Country: "Japanese"},
{Name: "Subaru", Country: "Japanese"},
{Name: "Mitsubishi", Country: "Japanese"},
{Name: "Suzuki", Country: "Japanese"},
{Name: "Mercedes-Benz", Country: "German"},
{Name: "BMW", Country: "German"},
{Name: "Audi", Country: "German"},
{Name: "Volkswagen", Country: "German"},
{Name: "Ford", Country: "American"},
{Name: "Chevrolet", Country: "American"},
{Name: "Hyundai", Country: "Korean"},
{Name: "Kia", Country: "Korean"},
{Name: "Peugeot", Country: "French"},
{Name: "Renault", Country: "French"},
{Name: "Volvo", Country: "Swedish"},
{Name: "Land Rover", Country: "British"},
{Name: "Isuzu", Country: "Japanese"},
}

// Create makes
for _, makeReq := range makes {
if _, err := vehicleService.CreateMake(makeReq); err != nil {
continue
}
}

log.Println("Initial vehicle data seeded successfully")
return nil
}
```

Routes

```
package routes

import (
"time"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/user-management/middleware"
"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/handlers"
authMiddleware "github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/middleware"
"github.com/gin-gonic/gin"
swaggerFiles "github.com/swaggo/files"
ginSwagger "github.com/swaggo/gin-swagger"
)

func SetupRoutes(
r *gin.Engine,
customerHandler *handlers.CustomerHandler,
partnerHandler *handlers.PartnerHandler,
agentHandler *handlers.AgentHandler,
valuationTypeHandler *handlers.ValuationTypeHandler,
branchHandler *handlers.BranchHandler,
vehicleInsuranceHandler *handlers.VehicleInsuranceHandler,
valuerHandler *handlers.ValuerHandler,
valuationRequestHandler *handlers.ValuationRequestHandler,
valuationCalculatorHandler *handlers.VehicleValuationHandler,
authServiceMiddleware *authMiddleware.AuthServiceMiddleware, // Updated parameter type
) {
// Add CORS middleware
r.Use(middleware.CORSMiddleware())

// Add logging middleware
r.Use(middleware.LoggingMiddleware())

// Health check (no authentication required)
r.GET("/health", func(c *gin.Context) {
c.JSON(200, gin.H{
"status": "healthy",
"service": "valuation-service",
"timestamp": time.Now(),
})
```

```
})

// Swagger documentation
r.GET("/docs/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))

// API routes
api := r.Group("/api/v1")

// Public routes (reference data that doesn't require authentication)
public := api.Group("/public")
{
// Public valuation types for dropdown/reference
public.GET("/valuation-types", valuationTypeHandler.GetActiveValuationTypes) //
GET /api/v1/public/valuation-types

// Public branches for dropdown/reference
public.GET("/branches", branchHandler.GetActiveBranches) // GET
/api/v1/public/branches

// Public valuers for dropdown/reference
public.GET("/valuers", valuerHandler.GetActiveValuers) // GET /api/v1/public/valuers
}

// Protected routes (using auth service)
protected := api.Group("/")
protected.Use(authServiceMiddleware.RequireAuth()) // Updated to use auth service
{
// Customer operations
customers := protected.Group("/customers")
{
// Basic CRUD operations
customers.GET("", customerHandler.GetAllCustomers) // GET /api/v1/customers?
limit=10&offset=0
customers.POST("", customerHandler.CreateCustomer) // POST /api/v1/customers
customers.GET("/:id", customerHandler.GetCustomerByID) // GET
/api/v1/customers/{id}
customers.PUT("/:id", customerHandler.UpdateCustomer) // PUT
/api/v1/customers/{id}
customers.DELETE("/:id", customerHandler.DeleteCustomer) // DELETE
/api/v1/customers/{id}
customers.GET("/search", customerHandler.SearchCustomers) // GET
/api/v1/customers/search?q=john&limit=10&offset=0
customers.GET("/status/:status", customerHandler.GetCustomersByStatus) // GET
/api/v1/customers/status/active?limit=10&offset=0
```

```
}

// Partner operations
partners := protected.Group("/partners")
{
// Basic CRUD operations
partners.GET("", partnerHandler.GetAllPartners) // GET /api/v1/partners?
limit=10&offset=0
partners.POST("", partnerHandler.CreatePartner) // POST /api/v1/partners
partners.GET("/:id", partnerHandler.GetPartnerByID) // GET /api/v1/partners/{id}
partners.PUT("/:id", partnerHandler.UpdatePartner) // PUT /api/v1/partners/{id}
partners.DELETE("/:id", partnerHandler.DeletePartner) // DELETE /api/v1/partners/{id}
partners.GET("/search", partnerHandler.SearchPartners) // GET
/api/v1/partners/search?q=company&limit=10&offset=0
partners.GET("/type/:type", partnerHandler.GetPartnersByType) // GET
/api/v1/partners/type/insurance?limit=10&offset=0
partners.GET("/status/:status", partnerHandler.GetPartnersByStatus) // GET
/api/v1/partners/status/active?limit=10&offset=0
partners.GET("/:id/agents", partnerHandler.GetPartnerWithAgents) // GET
/api/v1/partners/{id}/agents
}

// Agent operations
agents := protected.Group("/agents")
{
// Basic CRUD operations
agents.GET("", agentHandler.GetAllAgents) // GET /api/v1/agents?limit=10&offset=0
agents.POST("", agentHandler.CreateAgent) // POST /api/v1/agents
agents.GET("/:id", agentHandler.GetAgentByID) // GET /api/v1/agents/{id}
agents.PUT("/:id", agentHandler.UpdateAgent) // PUT /api/v1/agents/{id}
agents.DELETE("/:id", agentHandler.DeleteAgent) // DELETE /api/v1/agents/{id}
agents.GET("/search", agentHandler.SearchAgents) // GET /api/v1/agents/search?
q=agent&limit=10&offset=0
agents.GET("/type/:type", agentHandler.GetAgentsByType) // GET
/api/v1/agents/type/individual?limit=10&offset=0
agents.GET("/status/:status", agentHandler.GetAgentsByStatus) // GET
/api/v1/agents/status/active?limit=10&offset=0
agents.GET("/partner/:partner_id", agentHandler.GetAgentsByPartner) // GET
/api/v1/agents/partner/{partner_id}?limit=10&offset=0
agents.GET("/:id/valuation-requests", agentHandler.GetAgentWithValuationRequests)
// GET /api/v1/agents/{id}/valuation-requests
}

// Valuation Types operations
```

```
valuationTypes := protected.Group("/valuation-types")
{
// Basic CRUD operations
valuationTypes.GET("", valuationTypeHandler.GetAllValuationTypes) // GET
/api/v1/valuation-types?limit=10&offset=0
valuationTypes.POST("", valuationTypeHandler.CreateValuationType) // POST
/api/v1/valuation-types
valuationTypes.GET("/:id", valuationTypeHandler.GetValuationTypeByID) // GET
/api/v1/valuation-types/{id}
valuationTypes.PUT("/:id", valuationTypeHandler.UpdateValuationType) // PUT
/api/v1/valuation-types/{id}
valuationTypes.DELETE("/:id", valuationTypeHandler.DeleteValuationType) //
DELETE /api/v1/valuation-types/{id}
valuationTypes.GET("/search", valuationTypeHandler.SearchValuationTypes) // GET
/api/v1/valuation-types/search?q=insurance&limit=10&offset=0
valuationTypes.GET("/status/:status",
valuationTypeHandler.GetValuationTypesByStatus) // GET
/api/v1/valuation-types/status/active?limit=10&offset=0
valuationTypes.GET("/active", valuationTypeHandler.GetActiveValuationTypes) //
GET /api/v1/valuation-types/active?limit=10&offset=0
valuationTypes.GET("/name/:name",
valuationTypeHandler.GetValuationTypeByName) // GET
/api/v1/valuation-types/name/{name}
}

// Branch operations
branches := protected.Group("/branches")
{
// Basic CRUD operations
branches.GET("", branchHandler.GetAllBranches) // GET /api/v1/branches?
limit=10&offset=0
branches.POST("", branchHandler.CreateBranch) // POST /api/v1/branches
branches.GET("/:id", branchHandler.GetBranchByID) // GET /api/v1/branches/{id}
branches.PUT("/:id", branchHandler.UpdateBranch) // PUT /api/v1/branches/{id}
branches.DELETE("/:id", branchHandler.DeleteBranch) // DELETE
/api/v1/branches/{id}
branches.GET("/search", branchHandler.SearchBranches) // GET
/api/v1/branches/search?q=nairobi&limit=10&offset=0
branches.GET("/status/:status", branchHandler.GetBranchesByStatus) // GET
/api/v1/branches/status/active?limit=10&offset=0
branches.GET("/active", branchHandler.GetActiveBranches) // GET
/api/v1/branches/active?limit=10&offset=0
branches.GET("/location", branchHandler.GetBranchesByLocation) // GET
/api/v1/branches/location?location=nairobi&limit=10&offset=0
```

```
branches.GET("/name/:name", branchHandler.GetBranchByName) // GET
/api/v1/branches/name/{name}
branches.GET("/stats", branchHandler.GetBranchStats) // GET /api/v1/branches/stats
}

// Vehicle Insurance operations
vehicleInsurances := protected.Group("/vehicle-insurances")
{
// Basic CRUD operations
vehicleInsurances.GET("", vehicleInsuranceHandler.GetAllVehicleInsurances) // GET
/api/v1/vehicle-insurances?limit=10&offset=0
vehicleInsurances.POST("", vehicleInsuranceHandler.CreateVehicleInsurance) //
POST /api/v1/vehicle-insurances
vehicleInsurances.GET("/:id", vehicleInsuranceHandler.GetVehicleInsuranceByID) //
GET /api/v1/vehicle-insurances/{id}
vehicleInsurances.PUT("/:id", vehicleInsuranceHandler.UpdateVehicleInsurance) //
PUT /api/v1/vehicle-insurances/{id}
vehicleInsurances.DELETE("/:id", vehicleInsuranceHandler.DeleteVehicleInsurance) //
DELETE /api/v1/vehicle-insurances/{id}
vehicleInsurances.GET("/search", vehicleInsuranceHandler.SearchVehicleInsurances)
// GET /api/v1/vehicle-insurances/search?q=policy&limit=10&offset=0
vehicleInsurances.GET("/status/:status",
vehicleInsuranceHandler.GetVehicleInsurancesByStatus) // GET /api/v1/vehicle-
insurances/status/active?limit=10&offset=0
vehicleInsurances.GET("/customer/:customer_id",
vehicleInsuranceHandler.GetVehicleInsurancesByCustomer) // GET /api/v1/vehicle-
insurances/customer/{customer_id}?limit=10&offset=0
vehicleInsurances.GET("/company",
vehicleInsuranceHandler.GetVehicleInsurancesByCompany) // GET /api/v1/vehicle-
insurances/company?company=jubilee&limit=10&offset=0
vehicleInsurances.GET("/expiring",
vehicleInsuranceHandler.GetExpiringVehicleInsurances) // GET /api/v1/vehicle-
insurances/expiring?days=30&limit=10&offset=0
vehicleInsurances.GET("/policy/:policy_number",
vehicleInsuranceHandler.GetVehicleInsuranceByPolicyNumber) // GET
/api/v1/vehicle-insurances/policy/{policy_number}
vehicleInsurances.GET("/stats", vehicleInsuranceHandler.GetVehicleInsuranceStats) //
GET /api/v1/vehicle-insurances/stats
}

// Valuer operations
valuers := protected.Group("/valuers")
{
// Basic CRUD operations
```

```
valuers.GET("", valuerHandler.GetAllValuers) // GET /api/v1/valuers?limit=10&offset=0
valuers.POST("", valuerHandler.CreateValuer) // POST /api/v1/valuers
valuers.GET("/:id", valuerHandler.GetValuerByID) // GET /api/v1/valuers/{id}
valuers.PUT("/:id", valuerHandler.UpdateValuer) // PUT /api/v1/valuers/{id}
valuers.DELETE("/:id", valuerHandler.DeleteValuer) // DELETE /api/v1/valuers/{id}
valuers.GET("/search", valuerHandler.SearchValuers) // GET /api/v1/valuers/search?
q=john&limit=10&offset=0
valuers.GET("/status/:status", valuerHandler.GetValuersByStatus) // GET
/api/v1/valuers/status/active?limit=10&offset=0
valuers.GET("/active", valuerHandler.GetActiveValuers) // GET /api/v1/valuers/active?
limit=10&offset=0
valuers.GET("/location", valuerHandler.GetValuersByLocation) // GET
/api/v1/valuers/location?location=nairobi&limit=10&offset=0
valuers.GET("/phone/:phone", valuerHandler.GetValuerByPhone) // GET
/api/v1/valuers/phone/{phone}
valuers.GET("/:id/valuation-requests",
valuerHandler.GetValuerWithValuationRequests) // GET
/api/v1/valuers/{id}/valuation-requests
valuers.GET("/stats", valuerHandler.GetValuerStats) // GET /api/v1/valuers/stats
valuers.GET("/:id/workload", valuerHandler.GetValuerWorkload) // GET
/api/v1/valuers/{id}/workload
}

valuationRequests := protected.Group("/valuation-requests")
{
// Basic CRUD operations
valuationRequests.GET("/", valuationRequestHandler.GetAllValuationRequests) // GET
/api/v1/valuation-requests?limit=10&offset=0
valuationRequests.POST("/", valuationRequestHandler.CreateValuationRequest) //
POST /api/v1/valuation-requests
valuationRequests.GET("/:id", valuationRequestHandler.GetValuationRequestByID) //
GET /api/v1/valuation-requests/{id}
valuationRequests.PUT("/:id", valuationRequestHandler.UpdateValuationRequest) //
PUT /api/v1/valuation-requests/{id}
valuationRequests.DELETE("/:id", valuationRequestHandler.DeleteValuationRequest)
// DELETE /api/v1/valuation-requests/{id}

// Search and filter operations
valuationRequests.GET("/search",
valuationRequestHandler.SearchValuationRequests) // GET /api/v1/valuation-
requests/search?q=john&limit=10&offset=0
valuationRequests.GET("/status/:status",
valuationRequestHandler.GetValuationRequestsByStatus) // GET /api/v1/valuation-
requests/status/pending?limit=10&offset=0
```

```
// Relationship-based queries
valuationRequests.GET("/customer/:customer_id",
valuationRequestHandler.GetValuationRequestsByCustomer) // GET
/api/v1/valuation-requests/customer/{customer_id}?limit=10&offset=0
valuationRequests.GET("/partner/:partner_id",
valuationRequestHandler.GetValuationRequestsByPartner) // GET /api/v1/valuation-
requests/partner/{partner_id}?limit=10&offset=0
valuationRequests.GET("/agent/:agent_id",
valuationRequestHandler.GetValuationRequestsByAgent) // GET /api/v1/valuation-
requests/agent/{agent_id}?limit=10&offset=0
valuationRequests.GET("/valuer/:valuer_id",
valuationRequestHandler.GetValuationRequestsByValuer) // GET /api/v1/valuation-
requests/valuer/{valuer_id}?limit=10&offset=0

// Special operations
valuationRequests.PUT("/:id/assign-valuer",
valuationRequestHandler.AssignValuerToRequest) // PUT
/api/v1/valuation-requests/{id}/assign-valuer
valuationRequests.PUT("/:id/status", valuationRequestHandler.UpdateRequestStatus)
// PUT /api/v1/valuation-requests/{id}/status
}

valueCalculator := protected.Group("/value")
{

valueCalculator.POST("calculator", valuationCalculatorHandler.CalculateValuation) //
POST //calculate the value of the car.

}

// Admin only routes
admin := protected.Group("/admin")
admin.Use(authServiceMiddleware.RequireRole("admin", "super_admin")) // Updated
to use auth service
{
// Admin customer operations
// admin.GET("/customers/analytics", customerHandler.GetCustomerAnalytics)
// admin.POST("/customers/bulk-import", customerHandler.BulkImportCustomers)

}
}
}
```

Handlers

```
package handlers

import (
"fmt"

"github.com/Automobile-Association-of-Kenya/aa-core-system/internal/valuation-service/services"
"github.com/Automobile-Association-of-Kenya/aa-core-system/pkg/response"
"github.com/gin-gonic/gin"
"github.com/go-playground/validator/v10"
)

type AgentHandler struct {
agentService *services.AgentService
validator *validator.Validate
}

func NewAgentHandler(agentService *services.AgentService) *AgentHandler {
return &AgentHandler{
agentService: agentService,
validator: validator.New(),
}
}

// CreateAgent godoc
// @Summary Create a new valuation agent
// @Description Create a new valuation agent in the system
// @Tags agents
// @Accept json
// @Produce json
// @Param request body services.CreateAgentRequest true "Create agent request"
// @Success 201 {object} response.Response "Agent created successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /agents [post]
// @Security BearerAuth
func (h *AgentHandler) CreateAgent(c *gin.Context) {
var req services.CreateAgentRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
```

```
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

agent, err := h.agentService.CreateAgent(req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Agent created successfully", agent)
}

// GetAllAgents godoc
// @Summary Get all valuation agents
// @Description Get paginated list of all valuation agents
// @Tags agents
// @Produce json
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents [get]
func (h *AgentHandler) GetAllAgents(c *gin.Context) {
// Parse query parameters for pagination
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

agents, total, err := h.agentService.GetAllAgents(limit, offset)
```

```
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents")
return
}

data := map[string]interface{}{
"agents": agents,
"total": total,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents retrieved successfully", data)
}

// GetAgentByID godoc
// @Summary Get valuation agent by ID
// @Description Get specific valuation agent by ID
// @Tags agents
// @Produce json
// @Param id path string true "Agent ID"
// @Success 200 {object} response.Response "Agent retrieved successfully"
// @Failure 404 {object} response.Response "Agent not found"
// @Router /agents/{id} [get]
func (h *AgentHandler) GetAgentByID(c *gin.Context) {
agentID := c.Param("id")

agent, err := h.agentService.GetAgentByID(agentID)
if err != nil {
response.NotFound(c, "Agent not found")
return
}

response.Success(c, "Agent retrieved successfully", agent)
}

// UpdateAgent godoc
// @Summary Update valuation agent
// @Description Update an existing valuation agent
// @Tags agents
// @Accept json
// @Produce json
// @Param id path string true "Agent ID"
// @Param request body services.UpdateAgentRequest true "Update agent request"
```

```
// @Success 200 {object} response.Response "Agent updated successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /agents/{id} [put]
// @Security BearerAuth
func (h *AgentHandler) UpdateAgent(c *gin.Context) {
agentID := c.Param("id")

var req services.UpdateAgentRequest
if err := c.ShouldBindJSON(&req); err != nil {
response.BadRequest(c, "Invalid request body")
return
}

if err := h.validator.Struct(req); err != nil {
response.ValidationError(c, err.Error())
return
}

agent, err := h.agentService.UpdateAgent(agentID, req)
if err != nil {
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Agent updated successfully", agent)
}

// DeleteAgent godoc
// @Summary Delete valuation agent
// @Description Soft delete a valuation agent
// @Tags agents
// @Produce json
// @Param id path string true "Agent ID"
// @Success 200 {object} response.Response "Agent deleted successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 401 {object} response.Response "Unauthorized"
// @Router /agents/{id} [delete]
// @Security BearerAuth
func (h *AgentHandler) DeleteAgent(c *gin.Context) {
agentID := c.Param("id")

err := h.agentService.DeleteAgent(agentID)
if err != nil {
```

```
response.BadRequest(c, err.Error())
return
}

response.Success(c, "Agent deleted successfully", nil)
}

// SearchAgents godoc
// @Summary Search valuation agents
// @Description Search valuation agents by query string
// @Tags agents
// @Produce json
// @Param q query string true "Search query"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agent search completed successfully"
// @Failure 400 {object} response.Response "Bad request"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/search [get]
func (h *AgentHandler) SearchAgents(c *gin.Context) {
query := c.Query("q")
if query == "" {
response.BadRequest(c, "Search query is required")
return
}

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

agents, err := h.agentService.SearchAgents(query, limit, offset)
if err != nil {
```

```
response.InternalServerError(c, "Failed to search agents")
return
}

data := map[string]interface{}{
"agents": agents,
"query": query,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agent search completed successfully", data)
}

// GetAgentsByType godoc
// @Summary Get agents by type
// @Description Get valuation agents filtered by agent type
// @Tags agents
// @Produce json
// @Param type path string true "Agent type"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents by type retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/type/{type} [get]
func (h *AgentHandler) GetAgentsByType(c *gin.Context) {
agentType := c.Param("type")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}
```

```
agents, err := h.agentService.GetAgentsByType(agentType, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents by type")
return
}

data := map[string]interface{}{
"agents": agents,
"agent_type": agentType,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents by type retrieved successfully", data)
}

// GetAgentsByStatus godoc
// @Summary Get agents by status
// @Description Get valuation agents filtered by status
// @Tags agents
// @Produce json
// @Param status path string true "Agent status"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents by status retrieved successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/status/{status} [get]
func (h *AgentHandler) GetAgentsByStatus(c *gin.Context) {
status := c.Param("status")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
```

```
}

agents, err := h.agentService.GetAgentsByStatus(status, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents by status")
return
}

data := map[string]interface{}{
"agents": agents,
"status": status,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents by status retrieved successfully", data)
}

// GetAgentsByPartner godoc
// @Summary Get agents by partner
// @Description Get valuation agents filtered by partner ID
// @Tags agents
// @Produce json
// @Param partner_id path string true "Partner ID"
// @Param limit query int false "Number of items per page" default(10)
// @Param offset query int false "Number of items to skip" default(0)
// @Success 200 {object} response.Response "Agents by partner retrieved
successfully"
// @Failure 500 {object} response.Response "Internal server error"
// @Router /agents/partner/{partner_id} [get]
func (h *AgentHandler) GetAgentsByPartner(c *gin.Context) {
partnerID := c.Param("partner_id")

// Parse pagination parameters
limit := 10
offset := 0

if l := c.Query("limit"); l != "" {
if parsedLimit, err := parseInt(l); err == nil && parsedLimit > 0 {
limit = parsedLimit
}
}

if o := c.Query("offset"); o != "" {
```

```
if parsedOffset, err := parseInt(o); err == nil && parsedOffset >= 0 {
offset = parsedOffset
}
}

agents, err := h.agentService.GetAgentsByPartner(partnerID, limit, offset)
if err != nil {
response.InternalServerError(c, "Failed to retrieve agents by partner")
return
}

data := map[string]interface{}{
"agents": agents,
"partner_id": partnerID,
"limit": limit,
"offset": offset,
}

response.Success(c, "Agents by partner retrieved successfully", data)
}

// GetAgentWithValuationRequests godoc
// @Summary Get agent with valuation requests
// @Description Get valuation agent with their associated valuation requests
// @Tags agents
// @Produce json
// @Param id path string true "Agent ID"
// @Success 200 {object} response.Response "Agent with valuation requests retrieved
successfully"
// @Failure 404 {object} response.Response "Agent not found"
// @Router /agents/{id}/valuation-requests [get]
func (h *AgentHandler) GetAgentWithValuationRequests(c *gin.Context) {
agentID := c.Param("id")

agent, err := h.agentService.GetAgentWithValuationRequests(agentID)
if err != nil {
response.NotFound(c, "Agent not found")
return
}

response.Success(c, "Agent with valuation requests retrieved successfully", agent)
}

// Helper function to parse integer from string
```

```
func parseInt003(s string) (int, error) {
var result int
_, err := fmt.Sscanf(s, "%d", &result)
return result, err
}
```